SOC Automation Homelab

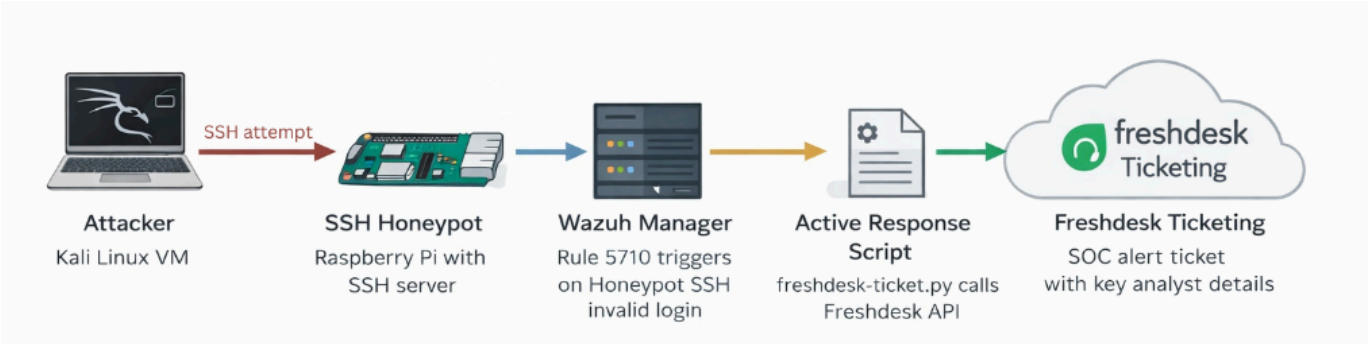
SSH Attack Detection & Automated Ticketing

Iwan Trythall · February 2026

Overview

This project was about building a working SOC detection and response pipeline from scratch, entirely on homelab hardware and free tooling. The goal was simple: simulate a real SSH brute force attack, have Wazuh catch it, and have the system automatically raise a pre-populated support ticket in Freshdesk, ready to be reviewed and sent to a client with zero manual input from the analyst.

The full pipeline ran end to end: attacker fires SSH attempts at a Raspberry Pi honeypot, Wazuh detects the failed logins and fires its active response script, and a structured SOC alert lands in Freshdesk within seconds. No clicking around, no copy and pasting logs, just automated, actionable intelligence.



Infrastructure Setup

The environment was deliberately minimal, the kind of thing you can run on a desk with hardware you likely already own.

Components

- Wazuh Manager - running on a local VM, acting as the SIEM and alerting engine
- Raspberry Pi - onboarded as a Wazuh agent, acting as the SSH honeypot
- Kali Linux VM - attacker machine, used to generate malicious SSH traffic
- Freshdesk - ticketing platform, receiving automated SOC alerts via API

Getting the Raspberry Pi enrolled as a Wazuh agent was straightforward, install the agent package, point it at the manager's IP, and register it. Once it appeared in the Wazuh dashboard as 'Active', the honeypot was live and forwarding

Detection Rules & Attack Simulation

Wazuh Rule 5710

Wazuh ships with a built in rule for this exact scenario, rule 5710 fires on any SSH login attempt against a non existent user. The rule tags the alert with MITRE ATT&CK techniques, GDPR/HIPAA compliance fields, and groups it under authentication_failed and invalid_login. No custom rule writing was needed here, which was the point, good SIEM tooling should handle commodity attacks out of the box.

The alert fired on `rule.firedtimes: 12` within the test window, with source IP `192.168.0.28` attempting to log in as `admin`, exactly what you'd expect from a credential stuffing or brute force run.

Attack Simulation

The attack was run from a Kali Linux VM on the same local network. SSH connection attempts were fired repeatedly at the Pi's IP, using invalid usernames and common passwords. From the attacker's perspective it was a basic recon/brute-force attempt. From Wazuh's perspective, every failed attempt was a log event, every log event was parsed by the sshd decoder, and repeated failures crossed the rule 5710 threshold in seconds.

The Wazuh dashboard confirmed 575 total alerts in the monitoring window, with 20 authentication failures and 72 authentication successes recorded across all agents, the spike in the alert level evolution graph matching exactly when the attack was launched.

The 72 authentication successes weren't from the attacker. Those were clean logins from other machines on the local network. That distinction is important in a real investigation, and it's exactly what the analyst's next actions in the ticket are there to prompt. Check for successful logins from the same source IP. In this case 192.168.0.28 had zero successes, only repeated failures, which points clearly to brute-force rather than someone using a credential that authenticated.

Automated Ticketing

Active Response Script

Wazuh's active response module was configured to execute `freshdesk-ticket.py` whenever rule 5710 fired. The script pulls the relevant fields directly from the alert, source IP, attempted username, hostname, rule ID, severity, and POSTs a structured ticket body to the Freshdesk API.

Ticket Output

The resulting Freshdesk ticket was designed to be analyst ready. Rather than dumping raw log lines, it presented the alert in a structured format with three clear sections:

- Summary - plain English description of what happened and what it may indicate
- Key Details - host, rule, severity, source IP, attempted username
- Analyst Next Actions - specific steps to take check for repeat failures, review for successful logins from the same source, escalate if brute force is confirmed

The raw log line was included as evidence at the bottom. Everything a level 1 analyst needs to triage the alert is already in the ticket, no context switching to the SIEM, no hunting through logs.

[SOC][Wazuh][SSH] Invalid Login Attempt | 192.168.0.28 → Honeypot

+ Add AI summary

New | Sentiment Negative

Created by **Iwan Trythall**

S **SOC lab** reported via the portal *Thu, 19 Feb 2026 at 3:59 PM*

🚨 SOC Alert: SSH Invalid Login Attempt

Summary

An SSH login attempt was made using an invalid or non-existent account. This may indicate reconnaissance or password guessing activity.

📌 Key Details

Host: Honeypot

Rule: 5710 — sshd invalid login

Severity: 5

Source IP: 192.168.0.28

Attempted Username: admin

✅ Analyst Next Actions

Check if this IP has repeated authentication failures

Review for successful logins from the same source

If repeated, escalate as suspected brute-force

📎 Evidence

```
Failed password for invalid user admin
```

Generated automatically by Wazuh — Freshdesk SOC Automation Lab

Show less

Results & Takeaways

The pipeline worked end to end. From the moment the attack started, Wazuh detected the invalid logins, the active response triggered the script, and a fully populated SOC alert ticket appeared in Freshdesk, all within seconds and with no manual intervention.

A few things stood out from building this:

- Wazuh's out of the box SSH rules are solid. Rule 5710 caught everything without any custom tuning.
- The active response framework is more flexible than it looks. Calling an external API from it is straightforward once you understand the alert JSON structure.
- Freshdesk ticket quality matters. A raw alert dump is useless to a client, structuring it with next actions is what makes it actionable.
- The MITRE ATT&CK tags Wazuh adds automatically are genuinely useful for reporting context without extra work.

The next logical step would be enriching the ticket with threat intelligence, running the source IP through something like AbuseIPDB before the ticket is created, so the analyst already knows whether the IP is a known malicious actor or first time offender.